APOSTILA DE COMPILADORES

Aula 14

Tabela de símbolos e escopos

Informações de declarações, resolução de identificadores e ambientes aninhados no compilador MINIC

Ciência da Computação — 2026/2
Professor: Alex Torquato Souza Carneiro
Projeto integrador: linguagem MINIC

Apresentação

Esta apostila acompanha a Aula 14 e marca a passagem da tradução dirigida por sintaxe para a análise semântica. O foco é compreender como o compilador registra declarações, organiza escopos e resolve cada uso de identificador no contexto correto.

A tabela de símbolos funciona como uma memória estruturada das construções declaradas no programa. Ela é preenchida durante a análise e consultada por fases posteriores, como a verificação de tipos e a geração de código.

Objetivos de aprendizagem

• explicar a finalidade e os campos de uma entrada da tabela de símbolos;

• distinguir escopo, ambiente e tabela de símbolos;

• implementar busca no escopo atual e nos escopos envolventes;

• identificar declarações duplicadas, identificadores não declarados e sombreamento;

• relacionar abertura e fechamento de escopos à AST e ao parser do MINIC.

Relação com as aulas anteriores

Nas Aulas 12 e 13, o scanner, o parser, a AST e as ações semânticas foram integrados. Agora, as ações associadas a declarações passam a produzir informações persistentes, que serão recuperadas quando o compilador encontrar identificadores em expressões, atribuições, chamadas e retornos.

1. O papel da tabela de símbolos

Uma tabela de símbolos é uma estrutura de dados que associa um nome às informações necessárias para interpretar uma construção da linguagem. O compilador pode criar uma entrada quando encontra uma declaração e consultá-la posteriormente quando encontra um uso do nome.

Informação	Exemplo no MINIC
Nome/lexema	total
Categoria	variável, função, parâmetro ou vetor
Tipo	inteiro, real, lógico ou retorno de função
Escopo	global, função ou bloco aninhado
Posição	linha e coluna da declaração
Atributos adicionais	tamanho do vetor, parâmetros, deslocamento ou endereço

O scanner pode reconhecer o lexema identificador, mas normalmente é o parser ou a análise semântica que determina se o nome aparece em uma declaração ou em uma expressão. Por isso, tabela de símbolos e AST trabalham em conjunto.

Exemplo de entrada conceitual

```
nome = total | categoria = variável | tipo = inteiro | escopo = função calcular | linha = 4 |
coluna = 9
```

2. Declarações e usos de identificadores

Uma declaração introduz um nome em determinado contexto. Um uso referencia esse nome. A análise semântica deve garantir que o uso encontre uma declaração visível e compatível com a regra de escopo da linguagem.

Fluxo básico

Momento	Ação
Declaração	verificar duplicidade no escopo atual e inserir o símbolo
Uso	buscar o nome a partir do escopo atual
Sucesso	anexar à AST a referência ou os atributos encontrados
Falha	emitir erro com nome e posição do uso

Exemplo no MINIC

```
inteiro total;
total = 10;
retorne total;
```

A primeira ocorrência cria uma entrada. As duas seguintes consultam a mesma entrada. A AST pode guardar o lexema e a posição; a análise semântica associa o identificador ao símbolo declarado.

3. Escopo e ambiente

Escopo é a região do programa em que uma declaração pode ser usada. Ambiente é o conjunto de tabelas de símbolos aplicáveis em um ponto específico do programa.

No escopo léxico, a resolução depende da estrutura textual do programa. Em uma sequência de escopos aninhados, a busca começa no ambiente mais interno e segue para os ambientes envolventes até encontrar o nome.

Conceito	Descrição
Escopo global	visível nos trechos permitidos pela linguagem
Escopo de função	parâmetros e declarações locais da função
Escopo de bloco	declarações entre chaves ou em um bloco equivalente
Ambiente	cadeia de tabelas acessíveis no ponto atual
Sombreamento	declaração interna oculta uma externa com o mesmo nome

Regra de busca

```
buscar(nome, atual):
1. procurar na tabela do escopo atual;
2. se não encontrar, consultar o escopo pai;
3. repetir até o escopo global;
4. se não encontrar, emitir “identificador não declarado”.
```

cap02c_AHO.qxd 26.10.07 12:39 Page 62

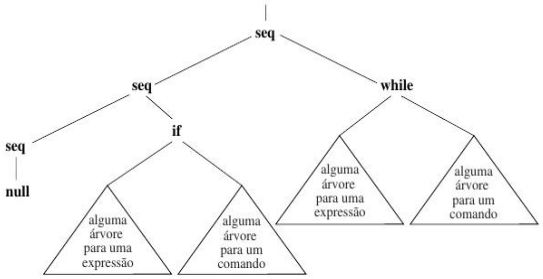


FIGURA 2.40 Parte de uma árvore sintática para uma lista de comandos consistindo em um comando if e um comando while.

Árvores sintáticas para expressões

Anteriormente, tratamos da precedência mais alta de * sobre + usando três não-terminais *expr*, *term* e *factor*. O número de não-terminais é exatamente um mais o número de níveis de precedência nas expressões, conforme sugerimos na Seção 2.2.6. Na Figura 2.39, temos dois operadores de comparação (< e <=) em um nível de precedência, além dos operadores normais + e *, de modo que acrescentamos um não-terminal adicional, chamado *add*.

A sintaxe abstrata nos permite agrupar operadores "semelhantes" para reduzir o número de casos e subclasses dos nós em uma implementação das expressões. Neste capítulo, "semelhante" significa que as regras de verificação de tipo e geração de código para os operadores são semelhantes. Por exemplo, normalmente os operadores + e * podem ser agrupados, uma vez que eles podem ser tratados da mesma maneira – seus requisitos em relação aos tipos dos operandos são iguais, e cada um deles resulta em uma única instrução de três endereços, que aplica um operador a dois valores. Em geral, o agrupamento de operadores na sintaxe abstrata é baseado nas necessidades das fases posteriores do compilador. A tabela na Figura 2.41 especifica a correspondência entre a sintaxe concreta e abstrata para vários dos operadores Java.

SINTAXE CONCRETA	SINTAXE ABSTRATA
=	**assign**
||	**cond**
&&	**cond**
== !=	**rel**
< <= >= >	**rel**
+ -	**op**
* / %	**op**
!	**not**
$-_{unário}$	**minus**
[ ]	**access**

FIGURA 2.41 Sintaxe concreta e abstrata para vários operadores Java.

Na sintaxe concreta, todos os operadores são associativos à esquerda, exceto o operador de atribuição =, que é associativo à direita. Os operadores em uma linha possuem a mesma precedência; ou seja, == e != têm a mesma precedência. As linhas estão em ordem de precedência crescente; por exemplo, == possui maior precedência do que os operadores && e =. O subscrito *unário* em $-_{unário}$ serve apenas para distinguir um sinal de subtração unário no início, como em -2, de um sinal de subtração binário, como em 2-a. O operador [ ] representa o acesso a arranjo, como em a[i].

A coluna de sintaxe abstrata especifica o agrupamento de operadores. O operador de atribuição = está em um grupo isolado. O grupo **cond** contém os operadores condicionais boolianos && e ||. O grupo **rel** contém os operadores de comparação relacionais nas linhas para == e <. O grupo **op** contém os operadores aritméticos como + e *. O operador de adição unário, a negação booliana e o acesso ao arranjo estão em grupos separados.

O mapeamento entre a sintaxe concreta e a abstrata, na Figura 2.41, pode ser implementado escrevendo-se um esquema de tradução. As produções para os não-terminais *expr*, *rel*, *add*, *term* e *factor* na Figura 2.39 especificam a sintaxe concreta

Figura 1 — Exemplo do livro-texto sobre escopos estruturados e encadeamento de ambientes. Fonte: Aho, Sethi e Ullman, Cap. 2.

4. Escopos aninhados e sombreamento

Considere o seguinte exemplo didático:

```
inteiro x; // escopo global
{
real x; // escopo interno: sombreia o global
x = 2.5; // refere-se ao x real
}
x = 1; // refere-se ao x inteiro global
```

O nome x possui duas declarações, mas cada uso é resolvido pelo escopo mais interno que contém uma declaração visível. O fechamento do bloco interno restaura o acesso ao símbolo global.

Pilhas de escopos

Uma implementação prática mantém uma pilha: ao entrar em um bloco, cria e empilha uma nova tabela; ao sair, remove a tabela do topo. A busca percorre a pilha do topo para a base.

Operação	Efeito
abrir_escopo()	cria tabela com referência ao pai
declarar(s)	insere s na tabela atual
buscar(s)	percorre tabela atual e pais
fechar_escopo()	descarta ou arquiva a tabela corrente

5. Implementação para o MINIC

A tabela de símbolos deve refletir as construções previstas na linguagem: variáveis, vetores, funções, parâmetros e, quando aplicável, informações de retorno.

Estrutura sugerida

```
Symbol {
nome: string;
categoria: VAR | VETOR | FUNCAO | PARAMETRO;
tipo: INTEIRO | REAL | LOGICO | VOID;
escopo: Scope;
linha: int;
coluna: int;
atributos: ...;
}
```

Integração com a AST

Ao visitar um nó de declaração, o analisador semântico insere o símbolo. Ao visitar um nó Identificador, consulta o ambiente e registra a associação encontrada. Essa associação permite que etapas posteriores conheçam o tipo, a categoria e a localização da entidade referenciada.

Exemplo de pseudocódigo

```
visitarDeclaracao(d):
se atual.contem(d.nome): erro duplicado
senão atual.inserir(criarSimbolo(d))

visitarIdentificador(i):
s = ambiente.buscar(i.nome)
se s == nulo: erro não declarado
senão i.simbolo = s
```

cap02c_AHO.qxd 26.10.07 12:39 Page 64

anteriormente poderia ser que $E_1.type$ e $E_2.type$ são convertidos para o mesmo tipo. Nesse caso, seria válido comparar, digamos, um inteiro com um float.

- *Sobrecarga.* O operador + em Java representa a adição quando aplicada a inteiros; ele significa concatenação quando aplicado a *strings*. Um símbolo é considerado *sobrecarregado* se tiver diferentes significados, cada um deles dependendo do seu contexto. Assim, + é sobrecarregado em Java. O significado de um operador sobrecarregado é determinado considerando-se os tipos conhecidos de seus operandos e resultados. Por exemplo, sabemos que o + em `z = x + y` é uma concatenação se soubermos que qualquer um dentre `x`, `y` ou `z` são do tipo string. Porém, se soubermos também que um deles é do tipo inteiro, então temos um erro de tipo, e não existe significado nesse uso de +.

2.8.4 CÓDIGO DE TRÊS ENDEREÇOS

Uma vez que as árvores sintáticas estão construídas, análise e síntese poderão ser feitas avaliando-se os atributos e executando-se os fragmentos de códigos nos nós da árvore. Ilustramos as possibilidades percorrendo as árvores sintáticas para gerar o código de três operandos. Especificamente, mostramos como escrever funções que processam a árvore sintática e, como efeito colateral, emitem o código necessário de três endereços.

Instruções de três endereços

O código de três endereços é uma seqüência de instruções na forma

$$x = y \textbf{ op } z$$

onde x, y e z são nomes, restrições ou temporários gerados pelo compilador; e **op** significa um operador.

Os arranjos são tratados usando as duas variantes de instruções a seguir:

$$x\,[\,y\,] = z$$

$$x = y\,[\,z\,]$$

A primeira coloca o valor de z no endereço $x[y]$, e a segunda coloca o valor de $y[z]$ no endereço x.

As instruções de três endereços são executadas em seqüência numérica, a menos que sejam forçadas a fazer de outra forma por um desvio condicional ou incondicional. Escolhemos as seguintes instruções para fluxo de controle:

`ifFalse` *x* `goto L`	se *x* é falso, execute em seguida a instrução rotulada com `L`
`ifTrue` *x* `goto L`	se *x* é verdadeiro, execute em seguida a instrução rotulada com `L`
`goto L`	execute em seguida a instrução rotulada com `L`

Um rótulo `L` pode ser conectado a qualquer instrução iniciando-a com um prefixo `L:`. Uma instrução pode ter mais de um rótulo.

Finalmente, precisamos de instruções que copiam um valor. A instrução de três endereços a seguir copia o valor de `y` para `x`:

$$x = y$$

Tradução de comandos

Os comandos são traduzidos para código de três endereços usando instruções de desvio para implementar o fluxo de controle através do comando. O leiaute da Figura 2.42 ilustra a tradução de **if** *expr* **then** $stmt_1$. O comando de desvio no leiaute

`ifFalse` *x* `goto` *after*

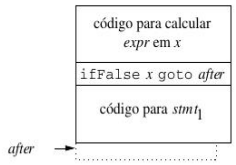


FIGURA 2.42 Leiaute de código para o comando if.

Figura 2 — Encadeamento de tabelas de símbolos apresentado no livro-texto. Fonte: Aho, Sethi e Ullman, Cap. 2.

A figura ilustra a ideia central: a tabela do bloco interno mantém uma referência para a tabela do bloco envolvente. Assim, nomes locais são encontrados primeiro, enquanto nomes externos continuam acessíveis quando não há declaração sombreando-os.

6. Erros semânticos relacionados

A tabela de símbolos fornece contexto para diagnosticar erros antes da geração de código.

Erro	Exemplo	Diagnóstico
Identificador não declarado	y = x + 1;	x não foi encontrado em nenhum escopo visível
Declaração duplicada	inteiro x; real x;	x já existe no escopo atual
Conflito de categoria	inteiro f; f();	o uso como função não corresponde à declaração
Parâmetro incompatível	f(1, 2);	quantidade ou tipos não correspondem à assinatura
Acesso fora do contexto	retorne x;	o uso não é válido no escopo ou função atual

Boas mensagens de erro

Toda mensagem deve indicar o tipo do problema, o lexema, a linha e a coluna. Quando possível, informe também a declaração que causou o conflito ou o escopo em que a busca foi realizada.

7. Atividades práticas

Atividade 1 — Modelo de entrada. Defina os campos de um registro de símbolo para variável, vetor, função e parâmetro. Justifique quais campos são comuns e quais são específicos.

Atividade 2 — Resolução. Para um programa com três blocos aninhados, construa as tabelas de símbolos e indique qual declaração é associada a cada uso de x, y e z.

Atividade 3 — Sombreamento. Explique o resultado da busca para um identificador declarado no escopo global e redeclarado em um bloco interno. Inclua o estado da pilha antes e depois do fechamento do bloco.

Atividade 4 — MINIC. Implemente abrir_escopo, fechar_escopo, declarar e buscar. Teste declarações duplicadas e identificadores não declarados.

Atividade 5 — Integração. Percorra a AST de uma função com parâmetros, variável local, atribuição e retorno. Produza uma tabela com todos os símbolos e as associações dos usos.

Checklist

- cada declaração é inserida no escopo correto;
- duplicidades são verificadas apenas no escopo atual;
- a busca respeita a regra do escopo mais interno;
- o fechamento de escopos restaura o ambiente anterior;
- os erros preservam posição e contexto;
- a AST recebe referências ou atributos semânticos necessários.

Síntese e leituras

A tabela de símbolos conecta declarações e usos. Em uma linguagem com escopo léxico, o ambiente pode ser representado por uma cadeia ou pilha de tabelas, com a regra de que o escopo mais interno tem prioridade. Essa estrutura prepara a verificação de tipos, a validação de chamadas e a geração de código.

No projeto MINIC, a implementação deve ser testada com programas que combinem escopos globais, funções, blocos aninhados, sombreamento e erros de resolução. O objetivo não é apenas armazenar nomes, mas preservar as informações necessárias para as próximas fases do compilador.

Leitura orientada

AHO, A. V.; SETHI, R.; ULLMAN, J. D. Compiladores: princípios, técnicas e ferramentas. Capítulo 2, Seção 2.7 — Tabelas de símbolos; e Capítulo 6, seção sobre análise semântica e ambientes.

Referência das imagens

Figuras reproduzidas para fins didáticos a partir do livro-texto enviado pelo docente: AHO, A. V.; SETHI, R.; ULLMAN, J. D. Compiladores: princípios, técnicas e ferramentas, Capítulo 2.